

Arquitectura Técnica

1. Enfoque arquitectónico

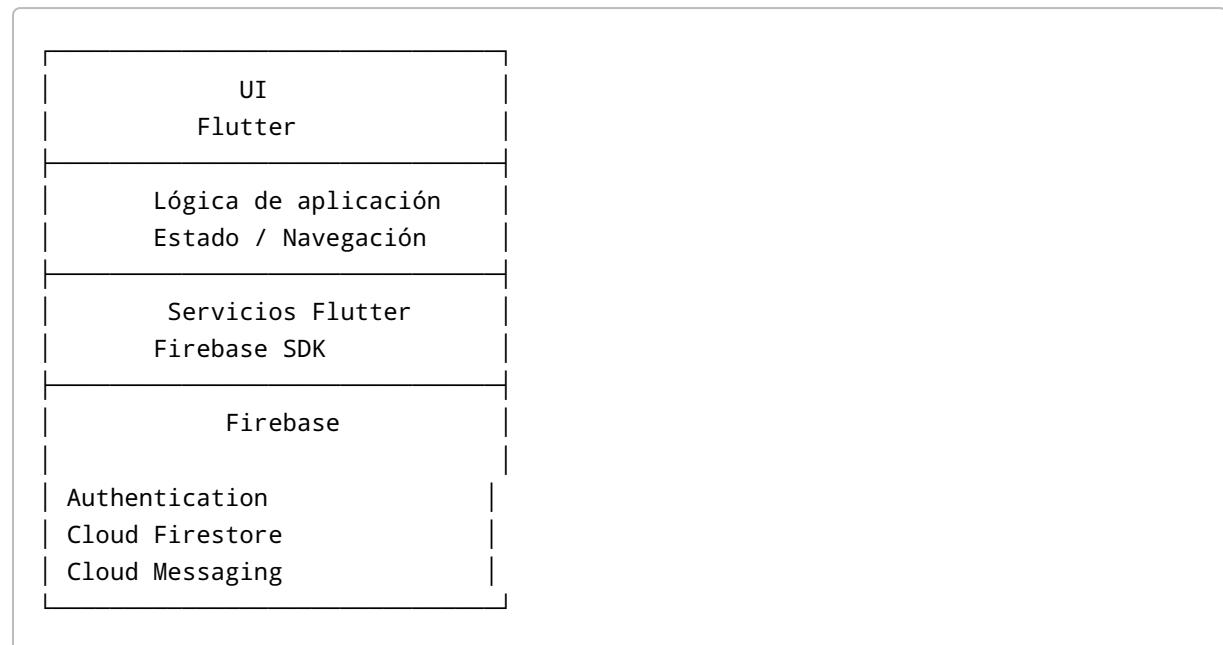
La aplicación utilizará una arquitectura móvil multiplataforma basada en **Flutter** para la construcción del frontend y **Firebase** como plataforma de servicios backend.

La elección busca reducir el tiempo de desarrollo y facilitar la creación de una única base de código para Android e iOS.

Esta decisión es coherente con el enfoque Lean Mobile Development, ya que permite construir rápidamente un MVP funcional para posteriormente validarlo con usuarios reales.

2. Arquitectura propuesta

La solución estará compuesta por:



3. Flutter

Flutter será responsable de la construcción de la aplicación móvil.

Sus principales responsabilidades serán:

- Construcción de interfaces.
- Navegación.

- Gestión de interacción.
- Presentación de datos.
- Validaciones del frontend.
- Comunicación con los servicios backend.

4. Firebase Authentication

Firebase Authentication permitirá implementar la gestión de usuarios.

El MVP podrá utilizarlo para:

- Crear cuentas.
- Iniciar sesión.
- Mantener sesiones.
- Cerrar sesión.
- Recuperar credenciales, según el alcance definido.

5. Cloud Firestore

Cloud Firestore será utilizado como base de datos para almacenar la información necesaria para el funcionamiento del MVP.

Las operaciones principales serán:

- Crear información.
- Consultar información.
- Actualizar información.
- Eliminar información cuando corresponda.

Las consultas deberán diseñarse de forma eficiente para contribuir al objetivo de rendimiento.

6. Firebase Cloud Messaging

Firebase Cloud Messaging permitirá implementar las notificaciones push.

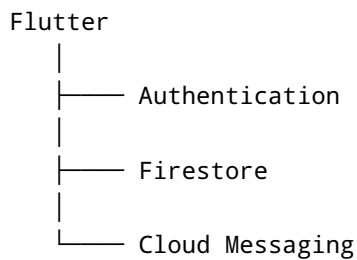
El flujo será:

```
Evento generado
  ↓
Firebase
  ↓
Cloud Messaging
  ↓
Dispositivo
  ↓
Notificación
```

↓
Usuario

7. Comunicación entre componentes

La aplicación Flutter se comunicará con Firebase mediante los SDK correspondientes.



Esto permitirá centralizar los servicios backend utilizados por el MVP.

8. Consideraciones de rendimiento

El diseño técnico tendrá como referencia el objetivo:

Tiempo de carga inferior a 1.5 segundos.

Para ello se considerarán:

- Consultas eficientes.
- Evitar consultas innecesarias.
- Reducción de datos transferidos.
- Optimización de imágenes.
- Carga progresiva.
- Uso adecuado del estado local.
- Minimización del trabajo realizado durante el arranque.

9. Consideraciones de seguridad

La solución deberá considerar:

- Autenticación de usuarios.
- Reglas de acceso a Firestore.
- Validación de datos.
- Protección de credenciales.
- Separación entre configuración pública de Firebase y secretos que no deben almacenarse en el repositorio.
- Uso de `.gitignore` para evitar publicar archivos sensibles.

10. Evolución de la arquitectura

La arquitectura inicial está orientada al MVP.

Después de las pruebas con usuarios podrán realizarse cambios de acuerdo con los resultados obtenidos.

El proceso será:

```
graph TD; A[Arquitectura inicial] --> B[Construcción MVP]; B --> C[Pruebas]; C --> D[Medición]; D --> E[Retroalimentación]; E --> F[Mejoras]; F --> G[Nueva iteración];
```

Arquitectura inicial
↓
Construcción MVP
↓
Pruebas
↓
Medición
↓
Retroalimentación
↓
Mejoras
↓
Nueva iteración

11. Estado del documento

Versión: 1.0

Estado: Propuesta inicial.

La arquitectura será actualizada durante el desarrollo del proyecto.